CYTOMACHIA

PROJECT TEAM

Sean Butterfield

- lead developer/designer
- buttersn@mail.uc.edu

PROJECT ADVISOR

Dr. Badri Vellambi

- badri.vellambi@uc.edu

CYTOMACHIA

κύτος
kytos
cell

μάχη
máchē
competition/battle

WHAT IS CYTOMACHIA?

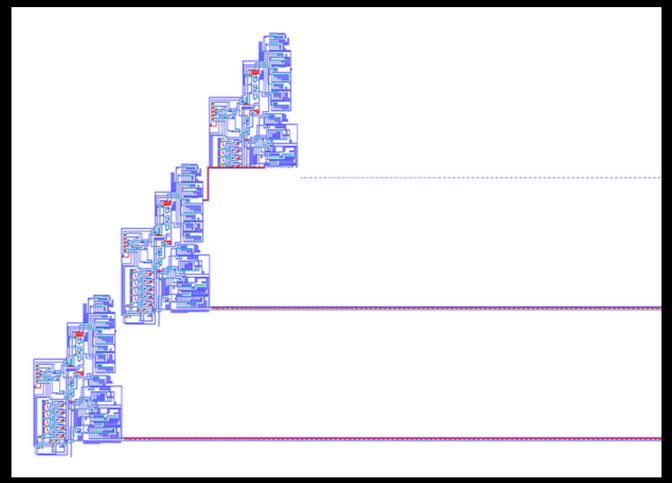
- Cytomachia is a 2D cellular automata engine that runs on a browser
- Open source, lightweight, free to use
- Computing is performed client-side, (no central server/datbase)

CA TLDR

- 2D Grid of cells that ‘evolves’ over time
- Each cell has a state: Alive (1) or Dead (0)
- The state of a cell is determined by its local conditions
- Cell states are determined by 2 rules:
  - Birth Rule: If an dead cell has N alive neighbors, the cell is born
  - Survival Rule: An alive cell requires N alive neighbors to stay alive
- Neighborhood: The shape of what a cell views as its neighbor
- Simple Rules → very complex results

CA BEGINNINGS

- John von Neumann and Stanislaw Ulam
- 1960's - 2D cell-based fluid simulation
  - Based on Newtonian fluid model
  - Cells represent the dx and dy of integral functions
- Shrink cells → shrink dx and dy, simulation approaches reality
- Simulating life and self-replication
  - Von Neumann Universal Constructor
    - 29 possible cell states

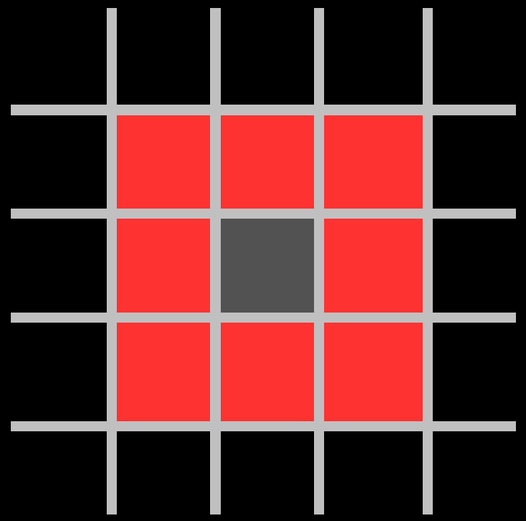


CONWAY'S GAME OF LIFE

- John Neuman, 1970
- Only 2 possible cell states
- Theorized self replicating in 1982
- Proven self replication in 2010, 2013, 2018
- Turing complete
- Ability to simulate itself: 'Life in life'

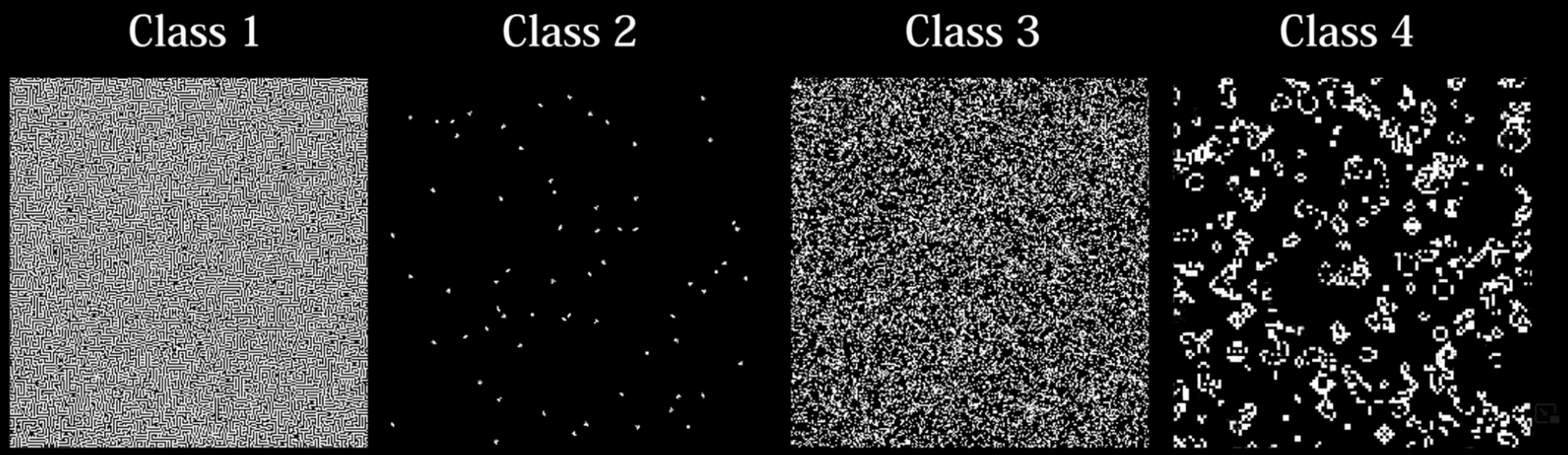
CONWAY'S GAME OF LIFE

- 2 states
- Birth Rule: 3 neighbors
- Survival Rule: 2-3 neighbors
- B3/S2..3
- Neighborhood is 8 adjacent cells (3x3)



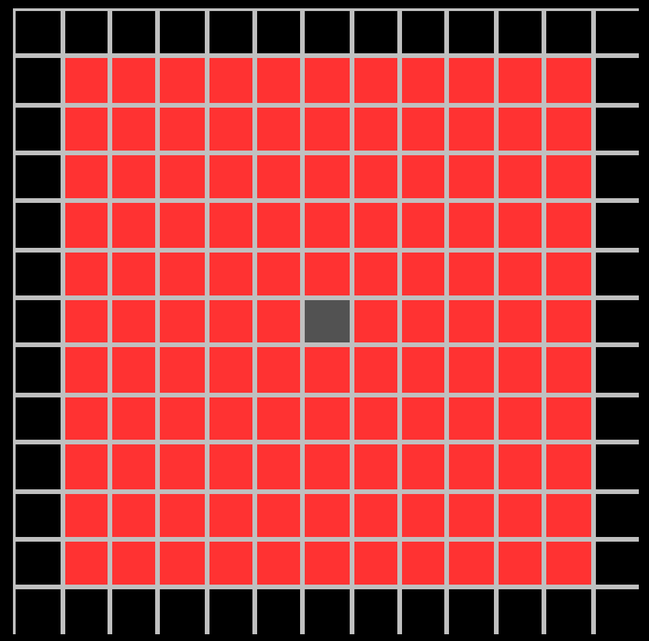
WOLFRAM CLASSIFICATION

- Class 1: Converges to a stable uniform structure
- Class 2: Converges to stable or oscillating structures
- Class 3: Chaos, no emergent patterns
- Class 4: Complex and ‘life-like’ behavior



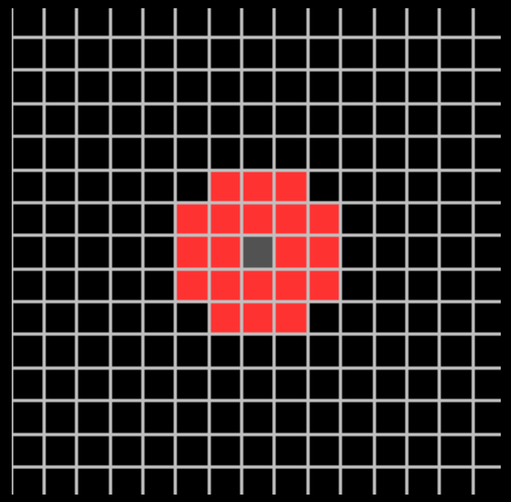
LARGER THAN LIFE

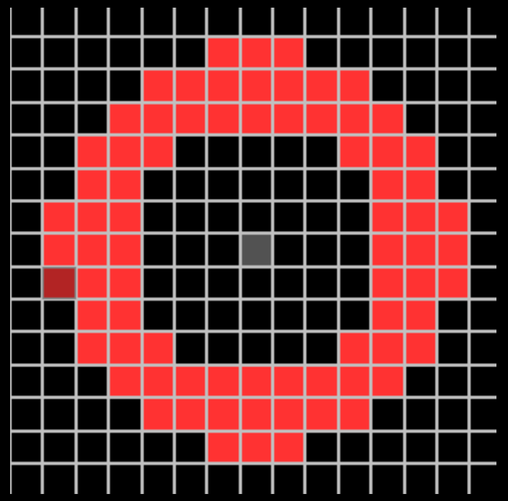
- Conway’s Game of Life: 3x3 → 8 possible neighbors
- ‘Bugs’: 11x11 → 120 possible neighbors
- Larger neighborhood → more information
- More information → more complex systems



MULTIPLE NEIGHBORHOODS

- Adding multiple neighborhood checks
  - Separate neighborhoods for birth and survival rules:
    - Double potential configurations for no performance impact





WEIGHTED KERNELS

- Supports weighted kernels
  - Minimal performance cost, orders of magnitude more possible configurations

0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	2	1	0	0	0	0
0	0	0	1	2	4	2	1	0	0	0
0	0	1	2	4	8	4	2	1	0	0
0	1	2	4	8		8	4	2	1	0
0	0	1	2	4	8	4	2	1	0	0
0	0	0	1	2	4	2	1	0	0	0
0	0	0	0	1	2	1	0	0	0	0
0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

HOW IT WORKS

- Handles ~3.7 million cells at up to 240 fps
- Compute and rendering cycles are ran on the GPU
  - Leverage the power of the GPU for max parallelism
- Simulation state is double-buffered (ping-pong) to eliminate race conditions during parallel access/edits
- User input is buffered, provides near instant feedback to user input, without affecting performance

TECHNOLOGIES

- HTML, TypeScript, SCSS
- Angular
- Material UI
- NodeJS
- RXJS
- Webgpu
- WGSL

Angular

node JS

Material UI

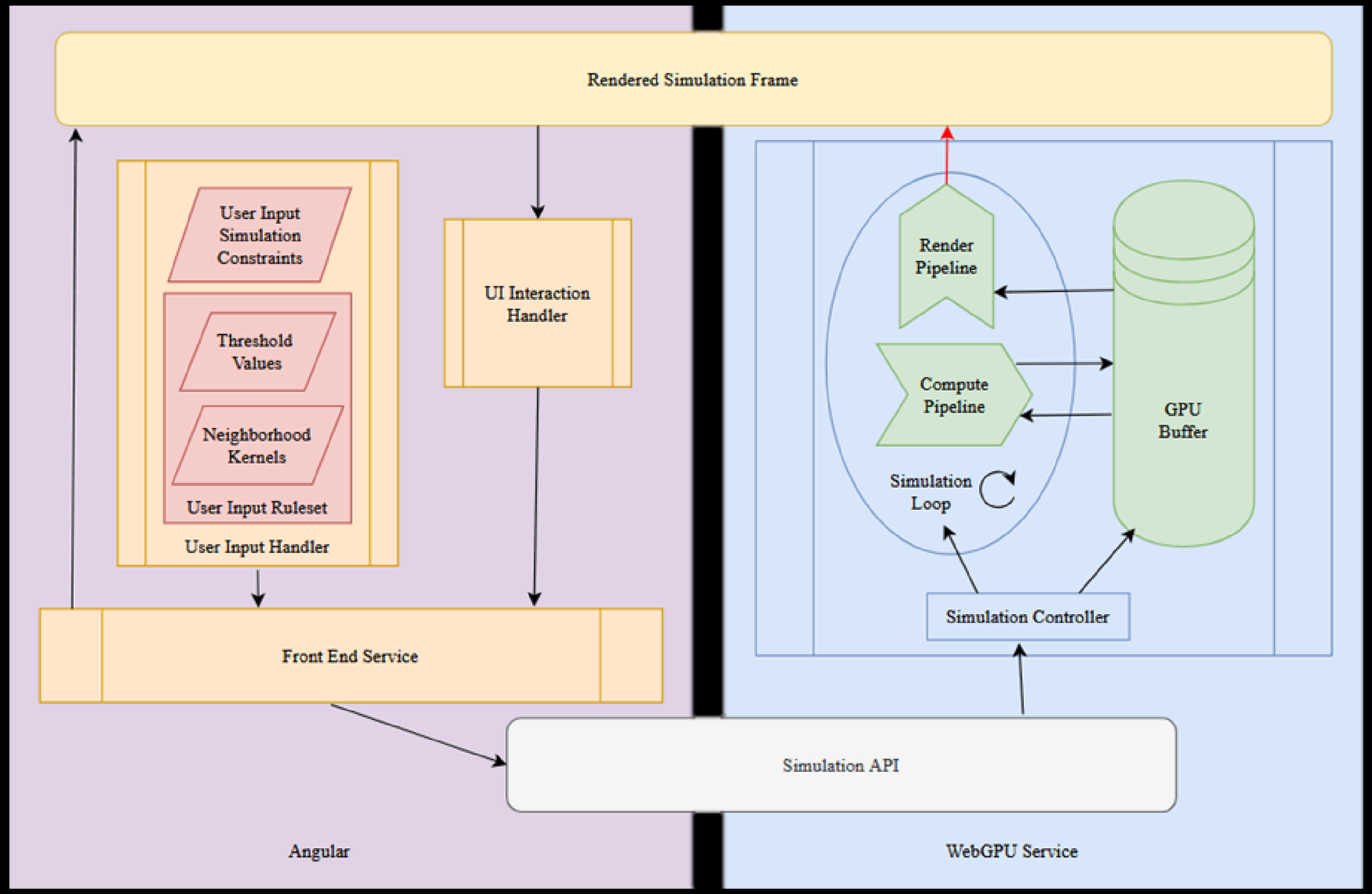
JS

HTML

CSS

RxJS

WebGPU


Rendered Simulation Frame
User Input Simulation Constraints
Threshold Values
Neighborhood Kernels
User Input Ruleset
User Input Handler
UI Interaction Handler
Front End Service
Simulation API
Angular
Render Pipeline
Compute Pipeline
Simulation Loop
GPU Buffer
Simulation Controller
WebGPU Service

CONSTRAINTS

- Scope: 1 Person team
- Performance
  - Run at a stable frame rate, state transitions feel smooth
  - Requires GPU to run
- Costs:
  - No central server or database
  - Client side computing
  - Cheap to host, easy to self host

CHALLENGES

- Handling User Input
  - CA configurations are often complex and unwieldy
  - Writing from RAM to VRAM is expensive
- Efficient compute shaders, and GPU memory management
  - Learning to write in parallel
  - Memory has to be manually managed
  - Buffers set up for efficient access/write by both shaders and Angular

MERITS/APPLICATION

- Sandbox, to learn and experiment with Cellular Automata
- Neural Cellular automata: AI gradient descent
- Biology:
  - Tumors, organ growth, disease spread, population growth
- Physics:
  - Fluid simulations, traffic flow simulations
- Cryptographic key generation, maze generation

FUTURE PLANS

- Improved UI and user input
  - Click and drag to edit kernels
  - Shaped brushes with configurable sizes
  - Control over seed generation thresholds and seed inputs
  - Edit mode for the simulation to toggle individual cells
- Allow for multiple competing cell types/’species’ and rules
  - GPU service is setup, but front end needs updating to handle multiple rulesets and cell ‘species’